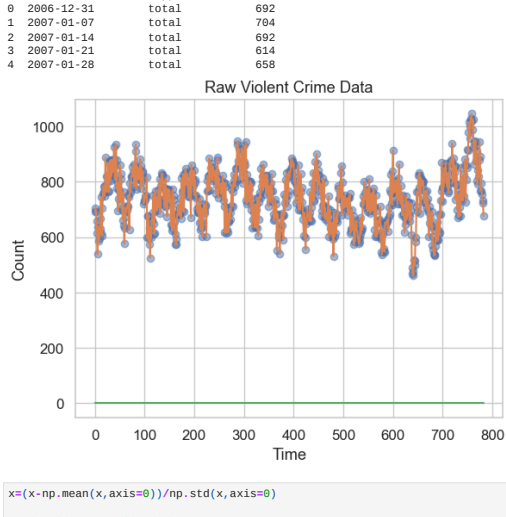
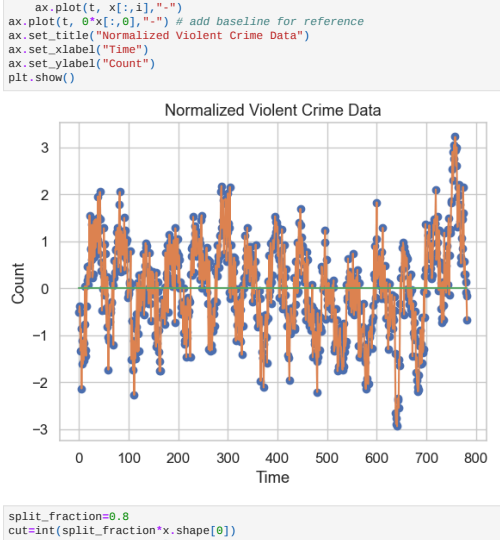
```
In [ ]: import warnings
        warnings.filterwarnings("ignore")
        import numpy as np
        import pandas as pd
        from tensorflow import keras
        from tensorflow.keras.layers import Dense, SimpleRNN, GRU, LSTM, Dropout, Bidirectional
        from sklearn.preprocessing import MinMaxScaler
        from sklearn.metrics import mean_squared_error
        import matplotlib.pyplot as plt
        from tensorflow.keras import initializers
        from tensorflow.keras import regularizers
        from sklearn.metrics import mean_squared_error,mean_absolute_percentage_error,mean_absolute_error
        from math import sqrt
        import seaborn as sns
```

```
In [ ]: df = pd.read_csv("./dataset/crimedata_week.csv")
        df.drop('offense_type', axis = 1)
        print(df.head())

        df = df.rename(columns={"week": "t", "Total_Crimes": "y"})
        df = df[["t","y"]]

        t=np.array([*range(0,df.shape[0])])
        x=np.array(df['y']).reshape(t.shape[0],1)
        feature_columns=[0] # columns to use as features
        target_columns=[0]  # columns to use as targets


        fig, ax = plt.subplots()
        for i in range(0,x.shape[1]):
            ax.plot(t, x[:,i],'o',alpha = 0.5)
            ax.plot(t, x[:,i],"-")
        ax.plot(t, 0*x[:,0],"-") # add baseline for reference
        ax.set_title("Raw Violent Crime Data")
        ax.set_xlabel("Time")
        ax.set_ylabel("Count")
        plt.show()
```

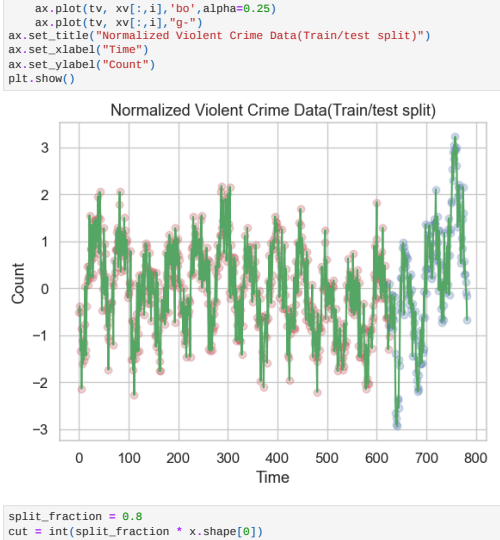
```
         week offense_type  Total_Crimes
0  2006-12-31        total           692
1  2007-01-07        total           704
2  2007-01-14        total           692
3  2007-01-21        total           614
4  2007-01-28        total           658
```


Raw Violent Crime Data

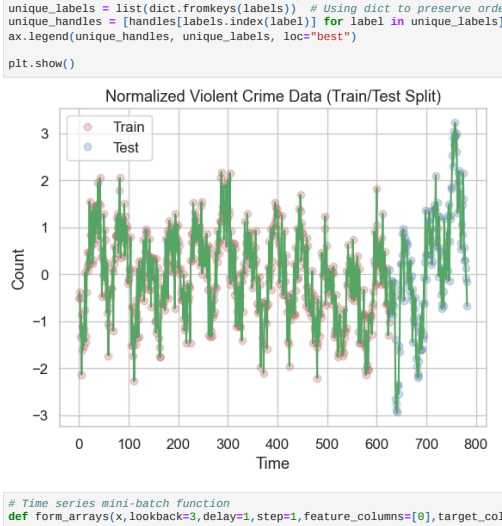
```
In [ ]: x=(x-np.mean(x,axis=0))/np.std(x,axis=0)

        # visualize normalized data
        fig, ax = plt.subplots()
        for i in range(0,x.shape[1]):
            ax.plot(t, x[:,i],'o')
            ax.plot(t, x[:,i],"-")
        ax.plot(t, 0*x[:,0],"-") # add baseline for reference
        ax.set_title("Normalized Violent Crime Data")
        ax.set_xlabel("Time")
        ax.set_ylabel("Count")
        plt.show()
```


Normalized Violent Crime Data

```
In [ ]: split_fraction=0.8
        cut=int(split_fraction*x.shape[0])
        tt=t[0:cut]; xt=x[0:cut]
        tv=t[cut:]; xv=x[cut:]

        # visualize normalized data
        fig, ax = plt.subplots()
        for i in range(0,x.shape[1]):
            ax.plot(tt, xt[:,i],'ro',alpha=0.25)
            ax.plot(tt, xt[:,i],"g-")
        for i in range(0,x.shape[1]):
            ax.plot(tv, xv[:,i],'bo',alpha=0.25)
            ax.plot(tv, xv[:,i],"g-")
        ax.set_title("Normalized Violent Crime Data(Train/test split)")
        ax.set_xlabel("Time")
        ax.set_ylabel("Count")
        plt.show()
```

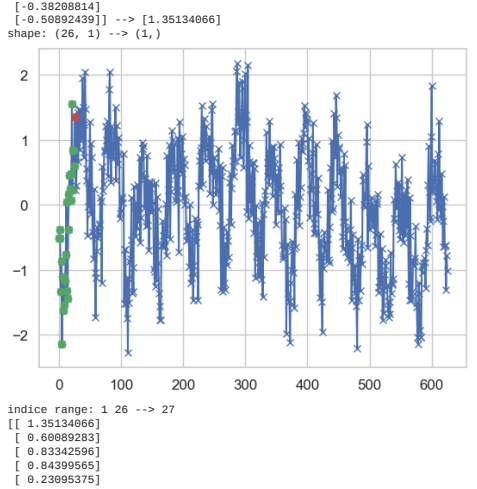

Normalized Violent Crime Data(Train/test split)

```
In [ ]: split_fraction = 0.8
        cut = int(split_fraction * x.shape[0])
        tt = t[0:cut]; xt = x[0:cut]
        tv = t[cut:]; xv = x[cut:]

        fig, ax = plt.subplots()
        for i in range(0, x.shape[1]):
            ax.plot(tt, xt[:,i], 'ro', alpha=0.25, label="Train" if i == 0 else "")
            ax.plot(tt, xt[:,i], "g-")
        for i in range(0, x.shape[1]):
            ax.plot(tv, xv[:,i], 'bo', alpha=0.25, label="Test" if i == 0 else "")
            ax.plot(tv, xv[:,i], "g-")

        ax.set_title("Normalized Violent Crime Data (Train/Test Split)")
        ax.set_xlabel("Time")
        ax.set_ylabel("Count")

        handles, labels = ax.get_legend_handles_labels()
        # We filter handles and labels to remove duplicates
        unique_labels = list(dict.fromkeys(labels))  # Using dict to preserve order
        unique_handles = [handles[labels.index(label)] for label in unique_labels]
        ax.legend(unique_handles, unique_labels, loc="best")

        plt.show()
```

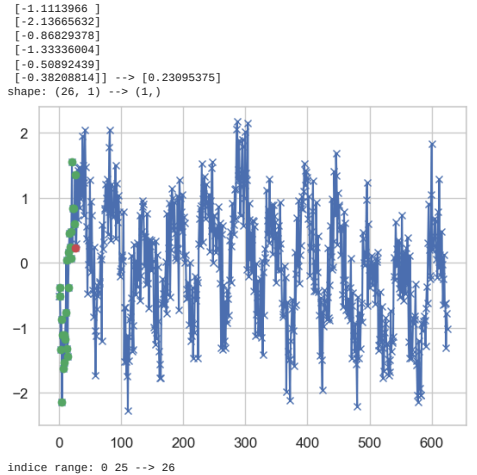

Normalized Violent Crime Data (Train/Test Split)

```
In [ ]: # Time series mini-batch function
        def form_arrays(x,lookback=3,delay=1,step=1,feature_columns=[0],target_columns=[0],unique=False,verbose=False):
            # verbose=True --> report and plot for debugging
            # unique=True --> don't re-sample:
            # x1,x2,x3 --> x4 then x4,x5,x6 --> x7 instead of x2,x3,x4 --> x5

            # initialize
            i_start=0; count=0;

            # initialize output arrays with samples
            x_out=[]
            y_out=[]

            # sequentially build mini-batch samples
            while i_start+lookback+delay< x.shape[0]:

                # define index bounds
                i_stop=i_start+lookback
                i_pred=i_stop+delay

                # report if desired
                if verbose and count<2: print("indice range:",i_start,i_stop,"-->",i_pred)

                # define arrays:
                # method-1: buggy due to indexing from left
                # numpy's slicing --> start:stop:step
                # xtmp=x[i_start:i_stop+1:steps]

                # method-2: non-vectorized but cleaner
                indices_to_keep=[]; j=i_stop
                while  j>=i_start:
                    indices_to_keep.append(j)
                    j=j-step

                # create mini-batch sample
                xtmp=x[indices_to_keep,:]    # isolate relevant indices
                xtmp=xtmp[:,feature_columns] # isolate desire features
                ytmp=x[i_pred,target_columns]
                x_out.append(xtmp); y_out.append(ytmp);

                # report if desired
                if verbose and count<2: print(xtmp, "-->",ytmp)
                if verbose and count<2: print("shape:",xtmp.shape, "-->",ytmp.shape)

                # PLOT FIRST SAMPLE IF DESIRED FOR DEBUGGING
                if verbose and count<2:
                    fig, ax = plt.subplots()
                    ax.plot(x,'b-')
                    ax.plot(x,'bx')
                    ax.plot(indices_to_keep,xtmp,'go')
                    ax.plot(i_pred*np.ones(len(target_columns)),ytmp,'ro')
                    plt.show()

                # UPDATE START POINT
                if unique: i_start+=lookback
                i_start+=1; count+=1

            return np.array(x_out),np.array(y_out)

        # training
        L=25; S=1; D=1
        Xt,Yt=form_arrays(xt,lookback=L,delay=D,step=S,feature_columns=feature_columns,target_columns=target_columns,unique=False,verbose=True)

        # validation
        Xv,Yv=form_arrays(xv,lookback=L,delay=D,step=S,feature_columns=feature_columns,target_columns=target_columns,unique=False,verbose=True)
```

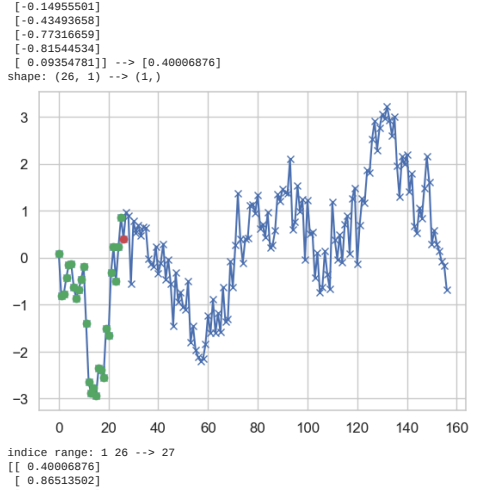
```
indice range: 0 25 --> 26
[[ 0.60989283]
 [ 0.83342596]
 [ 0.84399565]
 [ 0.23095375]
 [ 1.55216473]
 [ 0.47405657]
 [ 0.07248844]
 [ 0.24152344]
 [ 0.4529172 ]
 [ 0.16753563]
 [-0.38208814]
 [-1.43905692]
 [-1.32279036]
 [ 0.04069937]
 [-0.7625969 ]
 [-1.17481473]
 [-1.53418411]
 [-1.1113966 ]
 [-1.61874162]
 [-1.1113966 ]
 [-2.13665632]
 [-0.86829378]
 [-1.33336004]
 [-0.50892439]
 [-0.38208814]
 [-0.50892439]] --> [1.35134066]
shape: (26, 1) --> (1,)
```



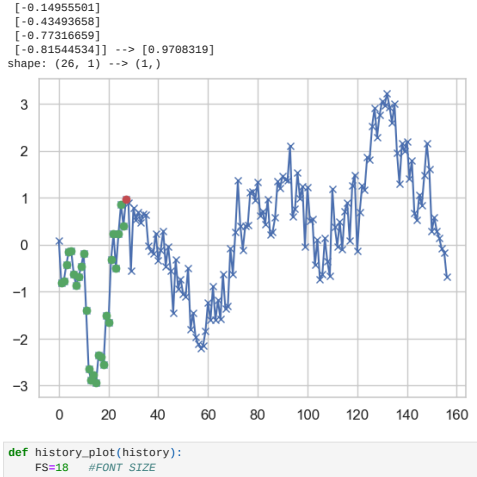
```
indice range: 1 26 --> 27
[[ 1.35134066]
 [ 0.60989283]
 [ 0.83342596]
 [ 0.84399565]
 [ 0.23095375]
 [ 1.55216473]
 [ 0.47405657]
 [ 0.07248844]
 [ 0.24152344]
 [ 0.4529172 ]
 [ 0.16753563]
 [-0.38208814]
 [-1.43905692]
 [-1.32279036]
 [ 0.04069937]
 [-0.7625969 ]
 [-1.17481473]
 [-1.53418411]
 [-1.1113966 ]
 [-1.61874162]
 [-1.1113966 ]
 [-2.13665632]
 [-0.86829378]
 [-1.33336004]
 [-0.50892439]
 [-0.38208814]] --> [0.23095375]
shape: (26, 1) --> (1,)
```



```
indice range: 0 25 --> 26
[[ 0.86513502]
 [ 0.23095375]
 [-0.49835471]
 [ 0.23095375]
 [-0.31867001]
 [-1.65045068]
 [-1.51364474]
 [-2.54887414]
 [-2.39032883]
 [-2.34805908]
 [-2.93995259]
 [-2.77983759]
 [-2.88718415]
 [-2.65457182]
 [-1.39677817]
 [-0.18126407]
 [-0.46664564]
 [-0.6780394 ]
 [-0.86829378]
 [-0.62519096]
 [-0.13898532]
 [-0.14955501]
 [-0.43493658]
 [-0.77316659]
 [-0.81544534]
 [ 0.09354781]] --> [0.40006876]
shape: (26, 1) --> (1,)
```



```
indice range: 1 26 --> 27
[[ 0.40006876]
 [ 0.86513502]
 [ 0.23095375]
 [-0.49835471]
 [ 0.23095375]
 [-0.31867001]
 [-1.65045068]
 [-1.51364474]
 [-2.54887414]
 [-2.39032883]
 [-2.34805908]
 [-2.93995259]
 [-2.77983759]
 [-2.88718415]
 [-2.65457182]
 [-1.39677817]
 [-0.18126407]
 [-0.46664564]
 [-0.6780394 ]
 [-0.86829378]
 [-0.62519096]
 [-0.13898532]
 [-0.14955501]
 [-0.43493658]
 [-0.77316659]
 [-0.81544534]] --> [0.9708319]
shape: (26, 1) --> (1,)
```



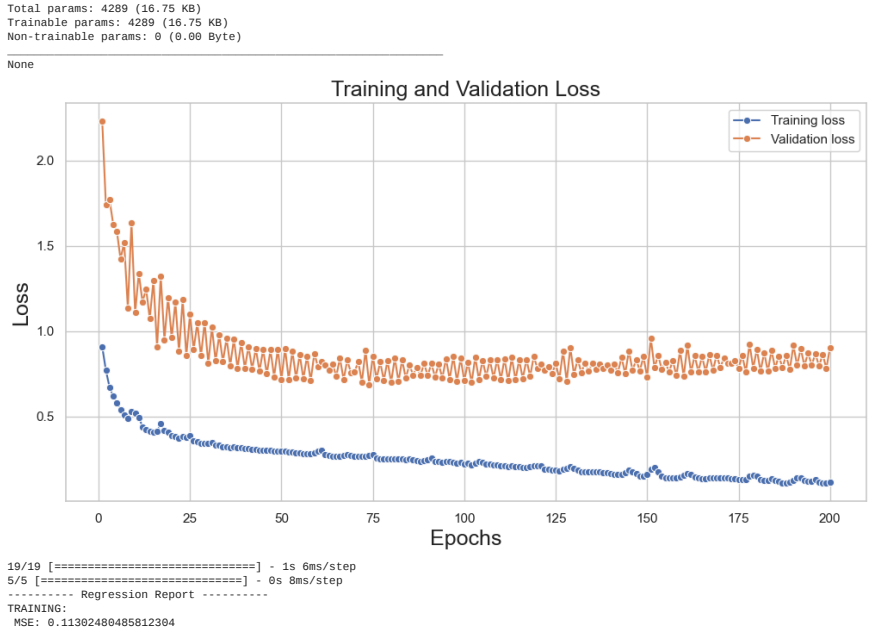
```
In [ ]: def history_plot(history):
            FS=18   #FONT SIZE
            # PLOTTING THE TRAINING AND VALIDATION LOSS
            history_dict = history.history
            loss_values = history_dict["loss"]
            val_loss_values = history_dict["val_loss"]
            epochs = range(1, len(loss_values) + 1)
            plt.plot(epochs, loss_values, "bo", label="Training loss")
            plt.plot(epochs, val_loss_values, "b", label="Validation loss")
            plt.title("Training and validation loss")
            plt.xlabel("Epochs")
            plt.ylabel("Loss")
            plt.legend()
            plt.show()

        # UTILITY FUNCTION
        def regression_report(yt,ytp,yv,yvp):
            print("---------- Regression report ----------")

            print("TRAINING:")
            print(" MSE:",mean_squared_error(yt,ytp))
            print(" MAE:",mean_absolute_error(yt,ytp))
            # print(" MAPE:",mean_absolute_percentage_error(Yt,Ytp))

            # PARITY PLOT
            fig, ax = plt.subplots()
            ax.plot(yt,ytp,'ro')
            ax.plot(yt,yt,'b-')
            ax.set(xlabel='y_data', ylabel='y_predicted',
                title='Training data parity plot (line y=x represents a perfect fit)')
            plt.show()

            # PLOT PART OF THE PREDICTED TIME-SERIES
            frac_plot=1.0
            upper=int(frac_plot*yt.shape[0]);
            # print(int(0.5*yt.shape[0]))
            fig, ax = plt.subplots()
            ax.plot(yt[0:upper],'b-')
            ax.plot(ytp[0:upper],'r-',alpha=0.5)
            ax.plot(ytp[0:upper],'ro',alpha=0.25)
            ax.set(xlabel='index', ylabel='y(t) (blue=actual & red=prediction)', title='Training: Time-series prediction')
            plt.show()


            print("VALIDATION:")
            print(" MSE:",mean_squared_error(yv,yvp))
            print(" MAE:",mean_absolute_error(yv,yvp))
            # print(" MAPE:",mean_absolute_percentage_error(Yt,Ytp))

            # PARITY PLOT
            fig, ax = plt.subplots()
            ax.plot(yv,yvp,'ro')
            ax.plot(yv,yv,'b-')
            ax.set(xlabel='y_data', ylabel='y_predicted',
                title='Validation data parity plot (line y=x represents a perfect fit)')
            plt.show()

            # PLOT PART OF THE PREDICTED TIME-SERIES
            upper=int(frac_plot*yv.shape[0])
            fig, ax = plt.subplots()
            ax.plot(yv[0:upper],'b-')
            ax.plot(yvp[0:upper],'r-',alpha=0.5)
            ax.plot(yvp[0:upper],'ro',alpha=0.25)
            ax.set(xlabel='index', ylabel='y(t) (blue=actual & red=prediction)', title='Validation: Time-series prediction')
            plt.show()
```

```
In [ ]: sns.set(style="whitegrid")

        def history_plot(history):
            FS = 18  # Font size
            history_dict = history.history
            loss_values = history_dict['loss']
            val_loss_values = history_dict['val_loss']
            epochs = range(1, len(loss_values) + 1)

            plt.figure(figsize=(12, 6))
            sns.lineplot(x=epochs, y=loss_values, marker='o', label='Training loss')
            sns.lineplot(x=epochs, y=val_loss_values, marker='o', label='Validation loss')
            plt.title('Training and Validation Loss', fontsize=FS)
            plt.xlabel('Epochs', fontsize=FS)
            plt.ylabel('Loss', fontsize=FS)
            plt.legend()
            plt.show()

        def regression_report(yt, ytp, yv, yvp):
            FS = 18  # Font size for plots
            sns.set_theme(context='notebook', style='whitegrid', palette='deep', font='sans-serif', font_scale=1.1, color_codes=True, rc=None)

            print("---------- Regression Report ----------")

            print("TRAINING:")
            print(" MSE:", mean_squared_error(yt, ytp))
            print(" MAE:", mean_absolute_error(yt, ytp))

            yt_flat = yt.flatten() if yt.ndim > 1 else yt
            ytp_flat = ytp.flatten() if ytp.ndim > 1 else ytp
            yv_flat = yv.flatten() if yv.ndim > 1 else yv
            yvp_flat = yvp.flatten() if yvp.ndim > 1 else yvp

            # Training Parity Plot
            plt.figure(figsize=(12, 7))
            sns.scatterplot(x=yt_flat, y=ytp_flat, color='magenta', alpha=0.6, edgecolor='black', marker='o')
            sns.lineplot(x=yt_flat, y=yt_flat, color='darkblue', label="y=x (Perfect fit)", linewidth=2.5)
            plt.title('Training Data Parity Plot', fontsize=FS)
            plt.xlabel('Actual Values', fontsize=FS)
            plt.ylabel('Predicted Values', fontsize=FS)
            plt.legend()
            plt.show()

            # Plot part of the Predicted Time-Series for Training
            plt.figure(figsize=(12, 7))
            sns.lineplot(x=np.arange(len(yt_flat)), y=yt_flat, label='Actual', color='blue', linewidth=1.2)
            sns.lineplot(x=np.arange(len(ytp_flat)), y=ytp_flat, label='Predicted', color='red', alpha=0.75, linewidth=1.2)
            plt.title('Training: Time-series Prediction', fontsize=FS)
            plt.xlabel('Time Index', fontsize=FS)
            plt.ylabel('Target Variable', fontsize=FS)
            plt.legend()
            plt.show()

            print("VALIDATION:")
            print(" MSE:", mean_squared_error(yv, yvp))
            print(" MAE:", mean_absolute_error(yv, yvp))

            # Validation Parity Plot
            plt.figure(figsize=(12, 7))
            sns.scatterplot(x=yv_flat, y=yvp_flat, color='magenta', alpha=0.6, edgecolor='black', marker='o')
            sns.lineplot(x=yv_flat, y=yv_flat, color='darkblue', label="y=x (Perfect fit)", linewidth=2.5)
            plt.title('Validation Data Parity Plot', fontsize=FS)
            plt.xlabel('Actual Values', fontsize=FS)
            plt.ylabel('Predicted Values', fontsize=FS)
            plt.legend()
            plt.show()

            # Plot part of the Predicted Time-Series for Validation
            plt.figure(figsize=(12, 7))
            sns.lineplot(x=np.arange(len(yv_flat)), y=yv_flat, label='Actual', color='blue', linewidth=2.5)
            sns.lineplot(x=np.arange(len(yvp_flat)), y=yvp_flat, label='Predicted', color='red', alpha=0.75, linewidth=2.5)
            plt.title('Validation: Time-series Prediction', fontsize=FS)
            plt.xlabel('Time Index', fontsize=FS)
            plt.ylabel('Target Variable', fontsize=FS)
            plt.legend()
            plt.show()
```

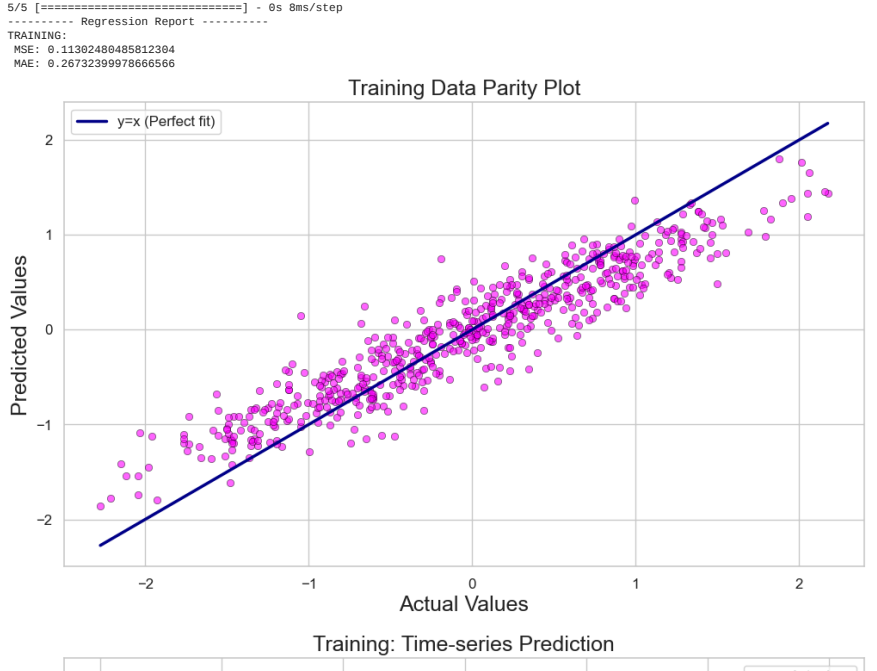
```
In [ ]: # RESHAPE INTO A DATA FRAME
        Xt1=Xt.reshape(Xt.shape[0],Xt.shape[1]*Xt.shape[2])
        Xv1=Xv.reshape(Xv.shape[0],Xv.shape[1]*Xv.shape[2])
```

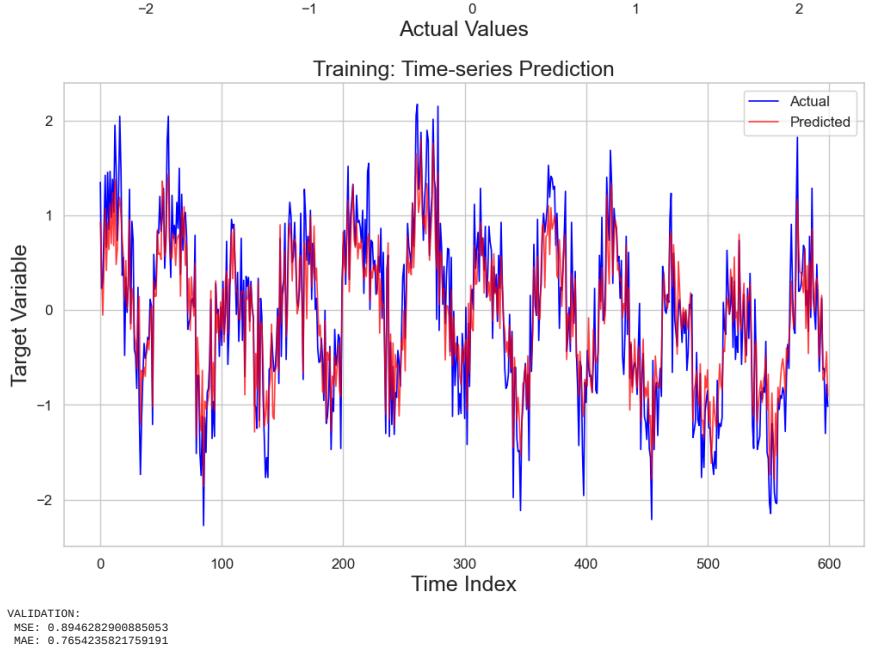
```
In [ ]: # # HYPERPARAMETERS
        optimizer="rmsprop"
        loss_function="MeanSquaredError"
        learning_rate=0.001
        numbers_epochs=200 #100
        L2=1e-4
        input_shape=(Xt.shape[1],Xt.shape[2])

        # # batch_size=1                       # stocastic training
        #batch_size=int(len(Xt1)/2.)    # mini-batch training
        #batch_size = int(len(Xt1) * 0.1)
        batch_size=len(Xt1)              # batch training

        # BUILD MODEL
        recurrent_hidden_units=64

        # CREATE MODEL
        model = keras.Sequential()

        # ADD RECURRENT LAYER

        # #COMMENT/UNCOMMENT TO USE RNN, LSTM,GRU
        #model.add(LSTM(
        #model.add(GRU(
        model.add(SimpleRNN(
        units=recurrent_hidden_units,
        return_sequences=False,
        input_shape=input_shape,
        recurrent_regularizer=regularizers.L2(L2),
        #recurrent_dropout=0.8,
        activation='tanh')
                  )

        # NEED TO TAKE THE OUTPUT RNN AND CONVERT TO SCALAR
        model.add(Dense(units=1, activation='linear'))

        # MODEL SUMMARY
        print(model.summary()); #print(x_train.shape,y_train.shape)
        # # print("initial parameters:", model.get_weights())

        # # COMPILING THE MODEL
        opt = keras.optimizers.RMSprop(learning_rate=learning_rate)
        #opt = keras.optimizers.SGD(learning_rate=learning_rate, momentum=0.0, nesterov=True)
        # opt = keras.optimizers.Adam(learning_rate=learning_rate, beta_1=0.9, beta_2=0.999)
        model.compile(optimizer=opt, loss=loss_function)

        # TRAINING YOUR MODEL
        history = model.fit(Xt,
                            Yt,
                            epochs=numbers_epochs,
                            batch_size=batch_size, verbose=False,
                            validation_data=(Xv, Yv))
        # History plot
        history_plot(history)

        # Predictions
        Ytp=model.predict(Xt)
        Yvp=model.predict(Xv)

        # REPORT
        regression_report(Yt,Ytp,Yv,Yvp)

        RNN_e_r = sqrt(mean_squared_error(Yt, Ytp))
```

```
Model: "sequential_60"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 simple_rnn_37 (SimpleRNN)   (None, 64)                4224

 dense_72 (Dense)            (None, 1)                 65

=================================================================
Total params: 4289 (16.75 KB)
Trainable params: 4289 (16.75 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
None
```

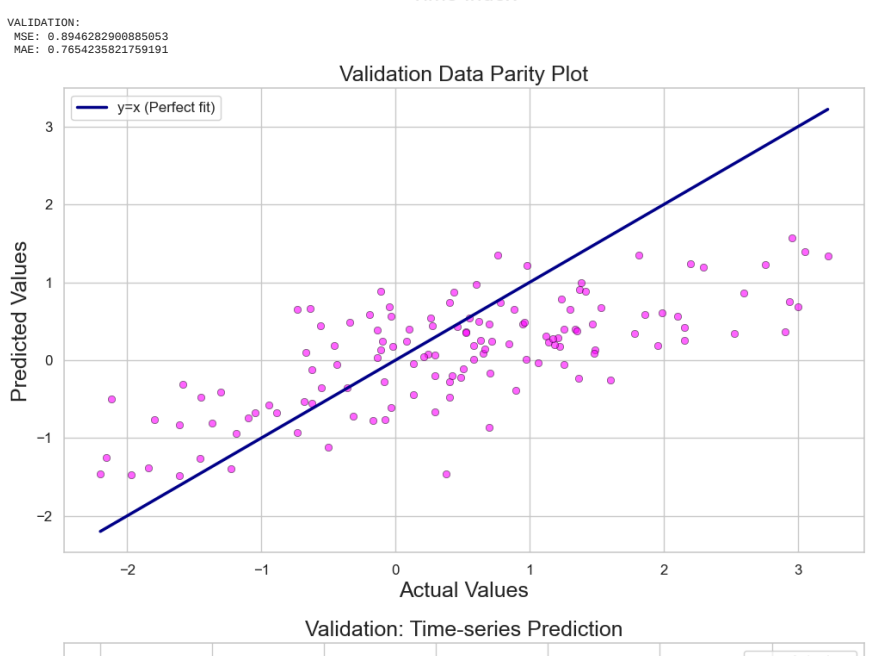

Training and Validation Loss

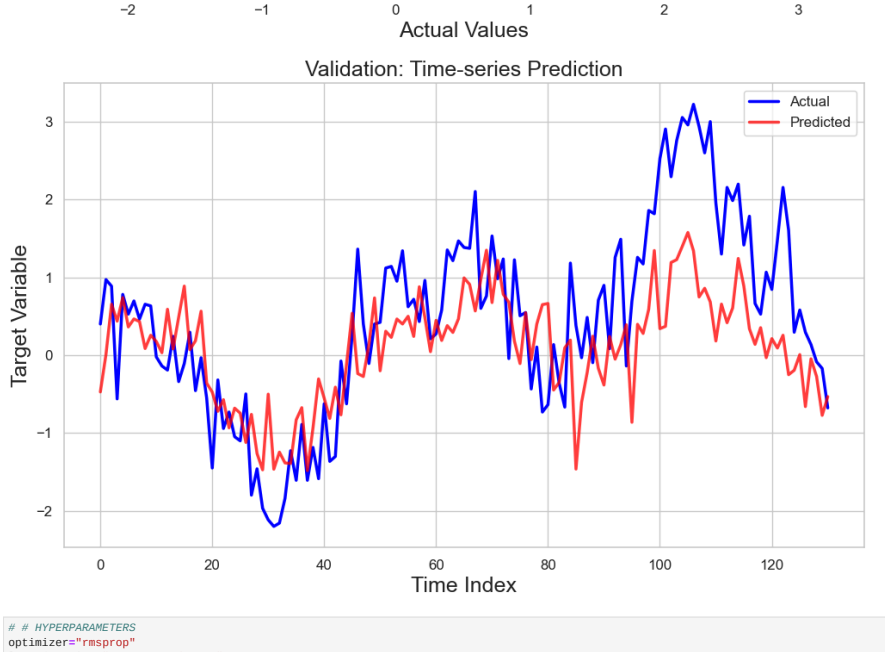
```
19/19 [==============================] - 1s 6ms/step
5/5 [==============================] - 0s 8ms/step
---------- Regression Report ----------
TRAINING:
 MSE: 0.11302480485812304
 MAE: 0.26732399978666566
```


Training Data Parity Plot


Training: Time-series Prediction

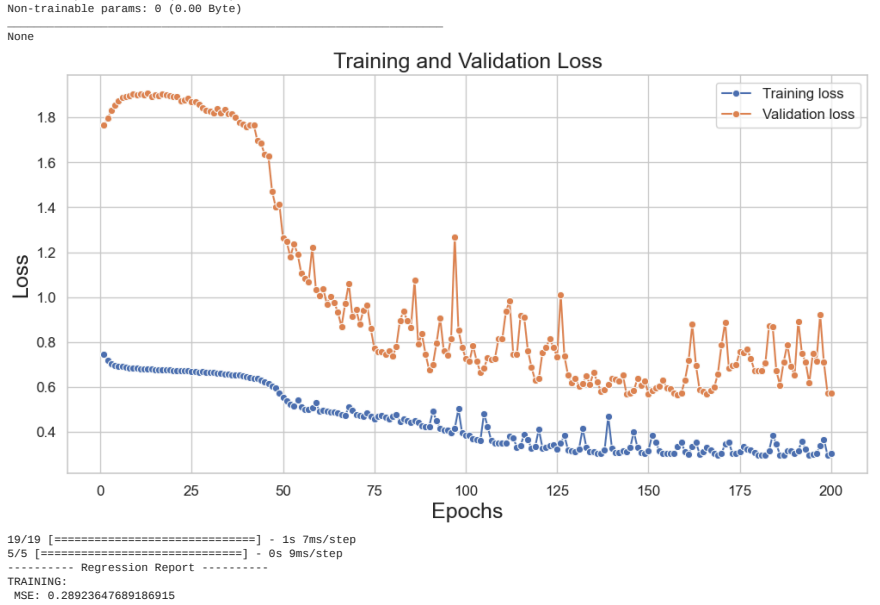
```
VALIDATION:
 MSE: 0.8946282900885053
 MAE: 0.7654235821759191
```


Validation Data Parity Plot

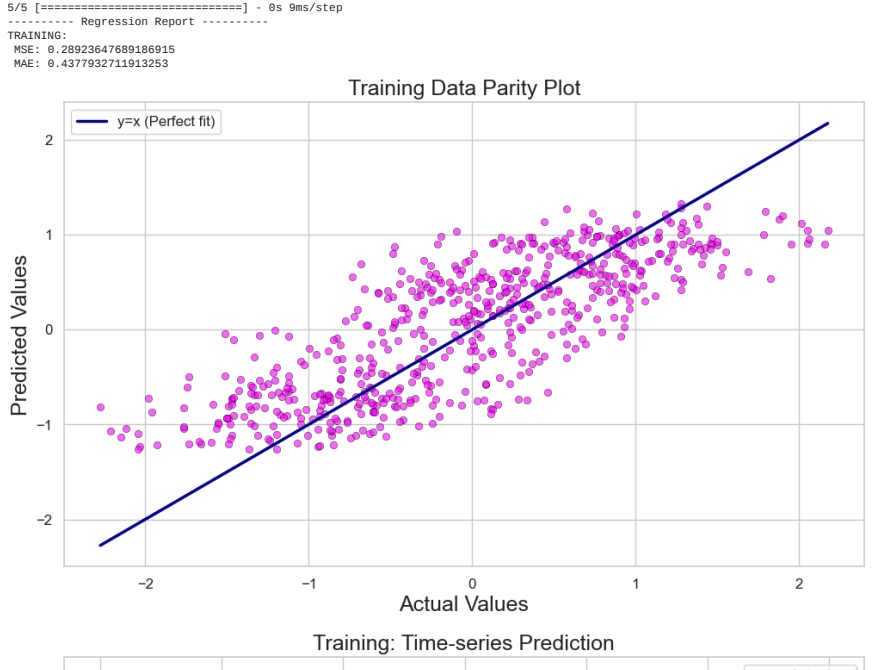

Validation: Time-series Prediction

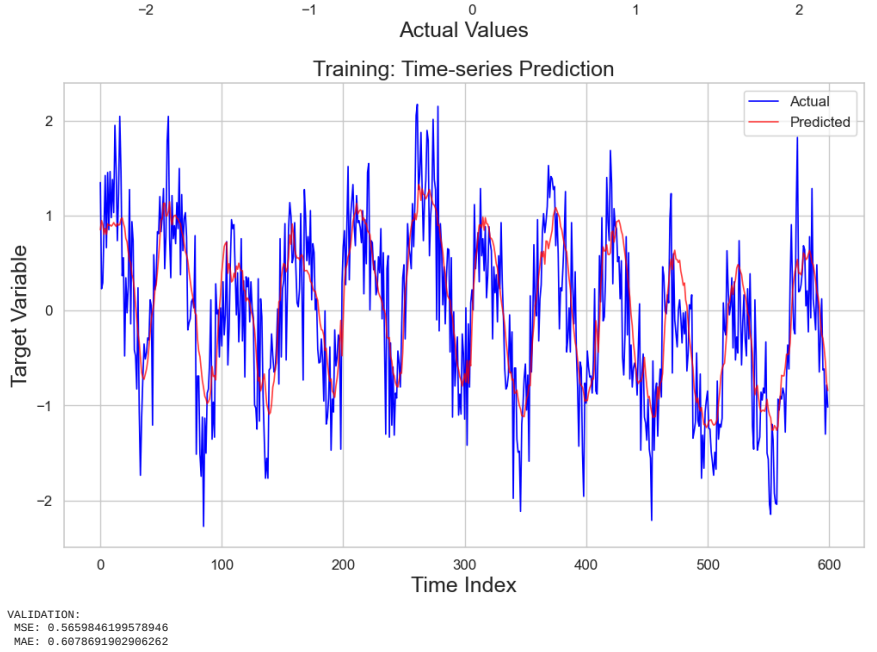
```
In [ ]: # # HYPERPARAMETERS
        optimizer="rmsprop"
        loss_function="MeanSquaredError"
        learning_rate=0.001
        numbers_epochs=200 #100
        L2=1e-4
        input_shape=(Xt.shape[1],Xt.shape[2])

        # # batch_size=1                       # stocastic training
        batch_size=int(len(Xt1)/2.)    # mini-batch training
        #batch_size = int(len(Xt1) * 0.1)
        # batch_size=len(Xt1)              # batch training

        # BUILD MODEL
        recurrent_hidden_units=64

        # CREATE MODEL
        model = keras.Sequential()

        # ADD RECURRENT LAYER

        # #COMMENT/UNCOMMENT TO USE RNN, LSTM,GRU
        #model.add(LSTM(
        model.add(GRU(
        #model.add(SimpleRNN(
        units=recurrent_hidden_units,
        return_sequences=False,
        input_shape=input_shape,
        recurrent_regularizer=regularizers.L2(L2),
        # recurrent_dropout=0.8,
        activation='relu')
                  )

        # NEED TO TAKE THE OUTPUT RNN AND CONVERT TO SCALAR
        model.add(Dense(units=1, activation='linear'))

        # MODEL SUMMARY
        print(model.summary()); #print(x_train.shape,y_train.shape)
        # # print("initial parameters:", model.get_weights())

        # # COMPILING THE MODEL
        opt = keras.optimizers.RMSprop(learning_rate=learning_rate)
        model.compile(optimizer=opt, loss=loss_function)

        # TRAINING YOUR MODEL
        history = model.fit(Xt,
                            Yt,
                            epochs=numbers_epochs,
                            batch_size=batch_size, verbose=False,
                            validation_data=(Xv, Yv))
        # History plot
        history_plot(history)

        # Predictions
        Ytp=model.predict(Xt)
        Yvp=model.predict(Xv)

        # REPORT
        regression_report(Yt,Ytp,Yv,Yvp)
        GRU_e_r = sqrt(mean_squared_error(Yt, Ytp))
```

```
Model: "sequential_61"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 gru_14 (GRU)                (None, 64)                12864

 dense_73 (Dense)            (None, 1)                 65

=================================================================
Total params: 12929 (50.50 KB)
Trainable params: 12929 (50.50 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
None
```

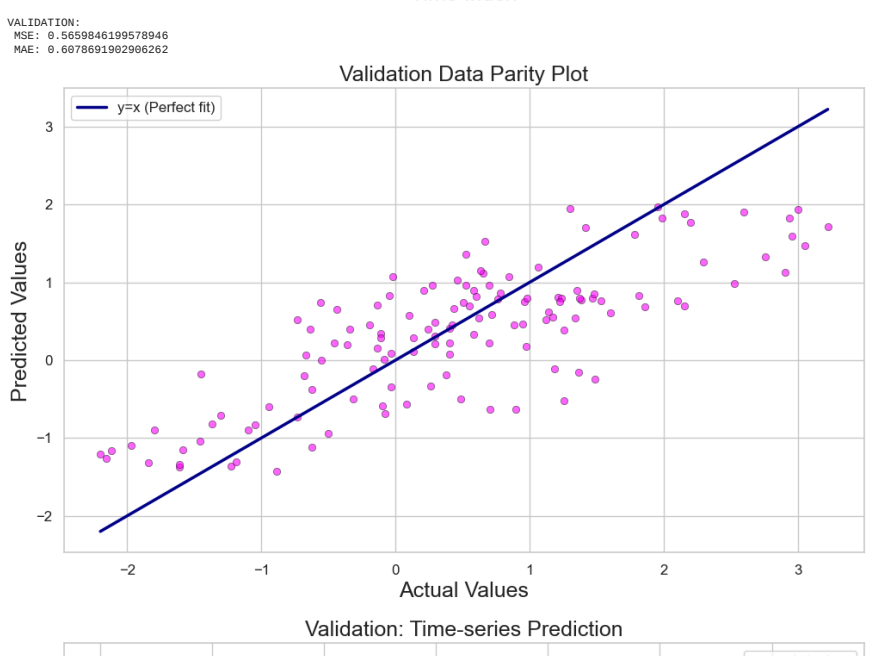

Training and Validation Loss

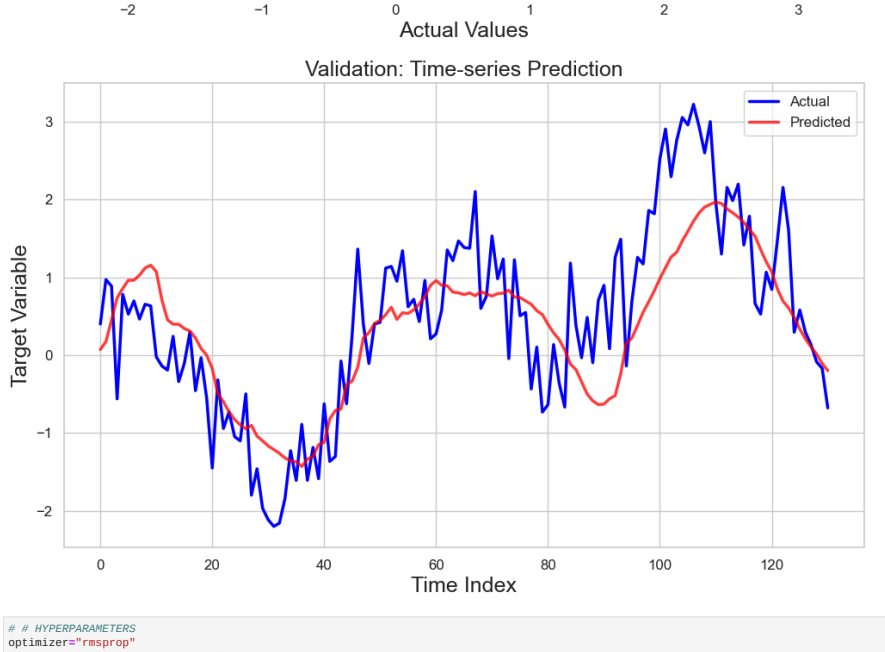
```
19/19 [==============================] - 1s 7ms/step
5/5 [==============================] - 0s 9ms/step
---------- Regression Report ----------
TRAINING:
 MSE: 0.28923647689186915
 MAE: 0.4377932711913253
```


Training Data Parity Plot


Training: Time-series Prediction

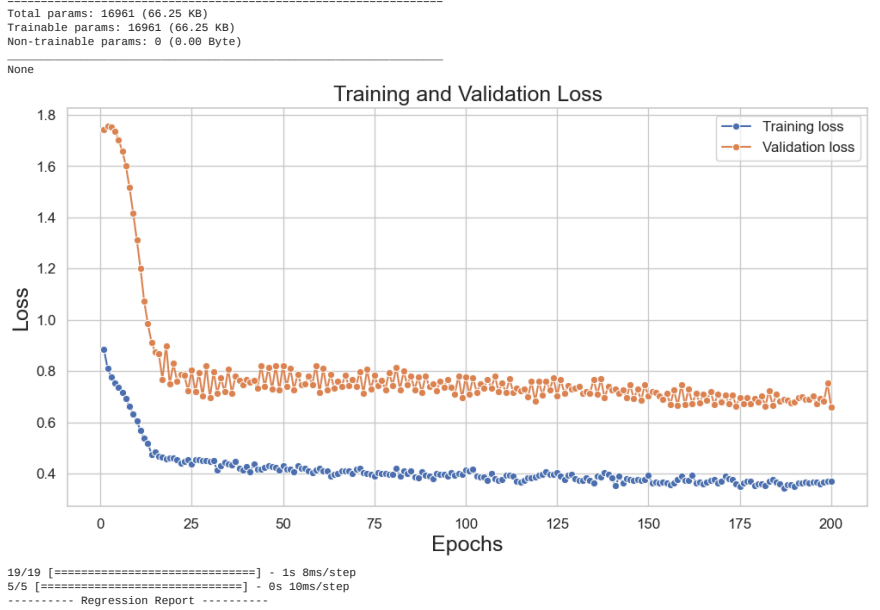
```
VALIDATION:
 MSE: 0.5659846199578946
 MAE: 0.6078691902906262
```


Validation Data Parity Plot

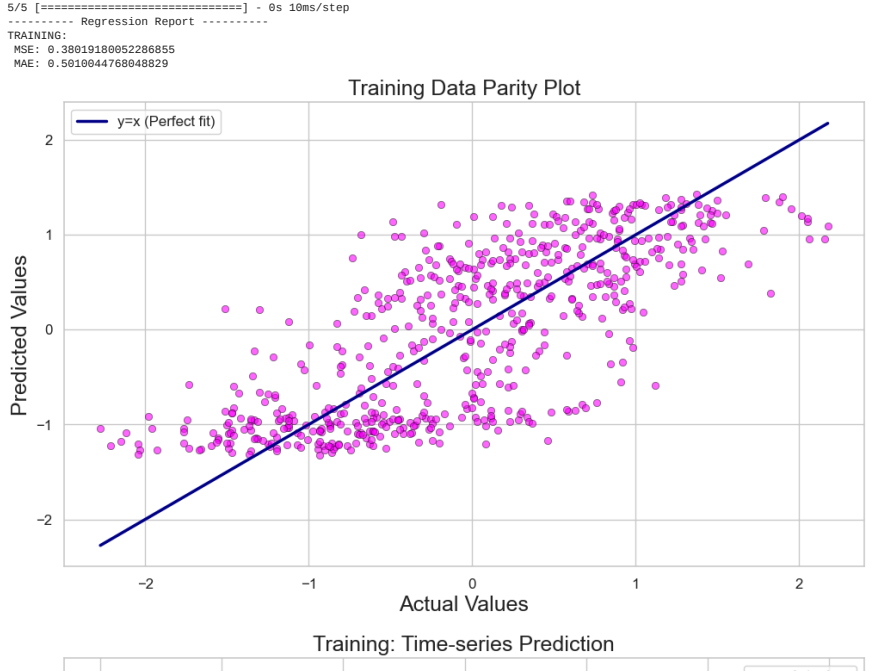

Validation: Time-series Prediction

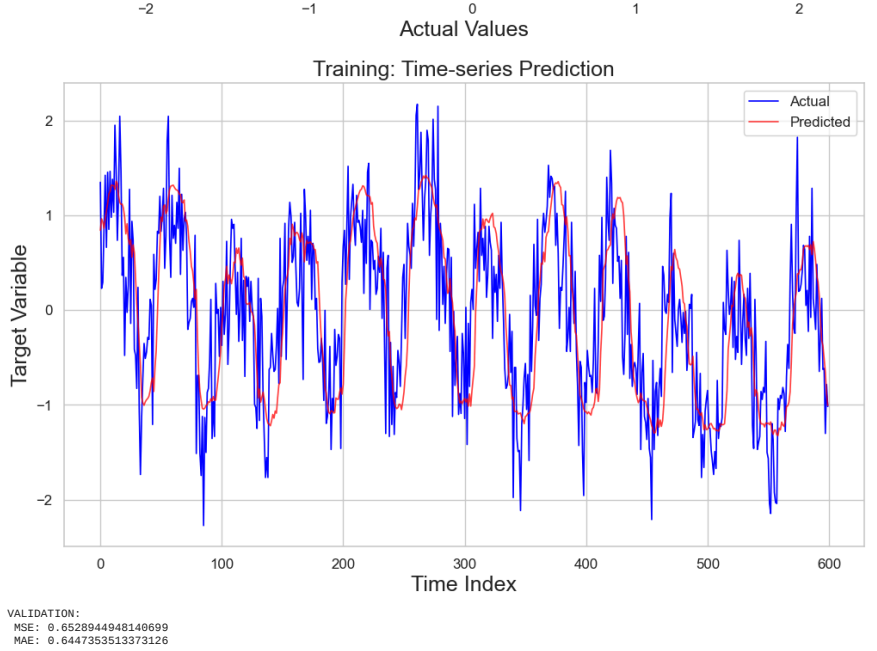
```
In [ ]: # # HYPERPARAMETERS
        optimizer="rmsprop"
        loss_function="MeanSquaredError"
        learning_rate=0.001
        numbers_epochs=200 #100
        L2=1e-4
        input_shape=(Xt.shape[1],Xt.shape[2])

        # # batch_size=1                       # stocastic training
        # # batch_size=int(len(x_train)/2.)    # mini-batch training
        batch_size=len(Xt1)              # batch training

        # BUILD MODEL
        recurrent_hidden_units=64

        # CREATE MODEL
        model = keras.Sequential()

        # ADD RECURRENT LAYER

        # #COMMENT/UNCOMMENT TO USE RNN, LSTM,GRU
        model.add(LSTM(
        #model.add(GRU(
        #model.add(SimpleRNN(
        units=recurrent_hidden_units,
        return_sequences=False,
        input_shape=input_shape,
        recurrent_regularizer=regularizers.L2(L2),
        recurrent_dropout=0.8,
        activation='tanh')
                  )

        # NEED TO TAKE THE OUTPUT RNN AND CONVERT TO SCALAR
        model.add(Dense(units=1, activation='linear'))

        # MODEL SUMMARY
        print(model.summary()); #print(x_train.shape,y_train.shape)
        # # print("initial parameters:", model.get_weights())

        # # COMPILING THE MODEL
        opt = keras.optimizers.RMSprop(learning_rate=learning_rate)
        model.compile(optimizer=opt, loss=loss_function)

        # TRAINING YOUR MODEL
        history = model.fit(Xt,
                            Yt,
                            epochs=numbers_epochs,
                            batch_size=batch_size, verbose=False,
                            validation_data=(Xv, Yv))
        # History plot
        history_plot(history)

        # Predictions
        Ytp=model.predict(Xt)
        Yvp=model.predict(Xv)

        # REPORT
        regression_report(Yt,Ytp,Yv,Yvp)
        LSTM_e_r = sqrt(mean_squared_error(Yt, Ytp))
```

```
Model: "sequential_62"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm_27 (LSTM)              (None, 64)                16896

 dense_74 (Dense)            (None, 1)                 65

=================================================================
Total params: 16961 (66.25 KB)
Trainable params: 16961 (66.25 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
None
```

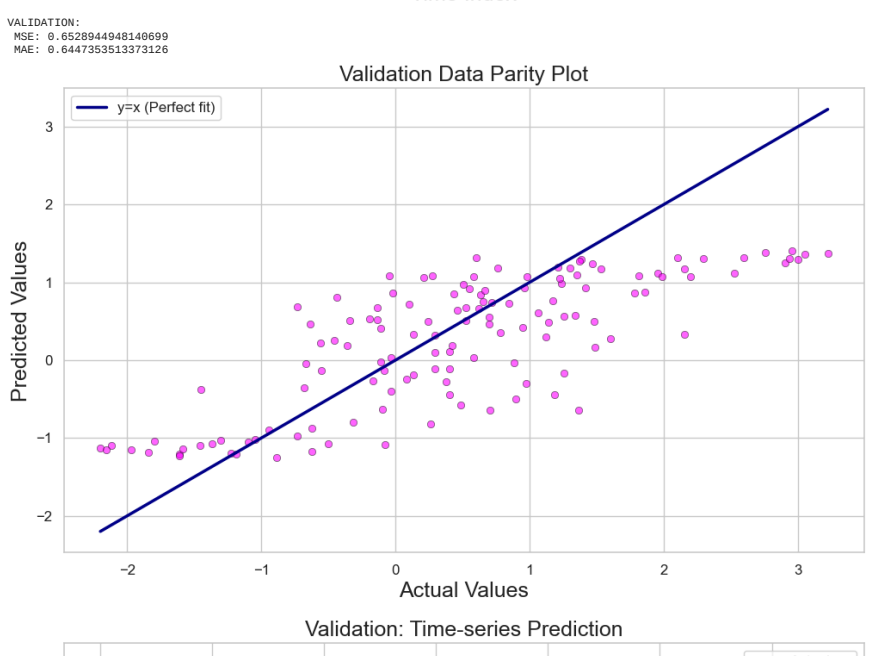

Training and Validation Loss

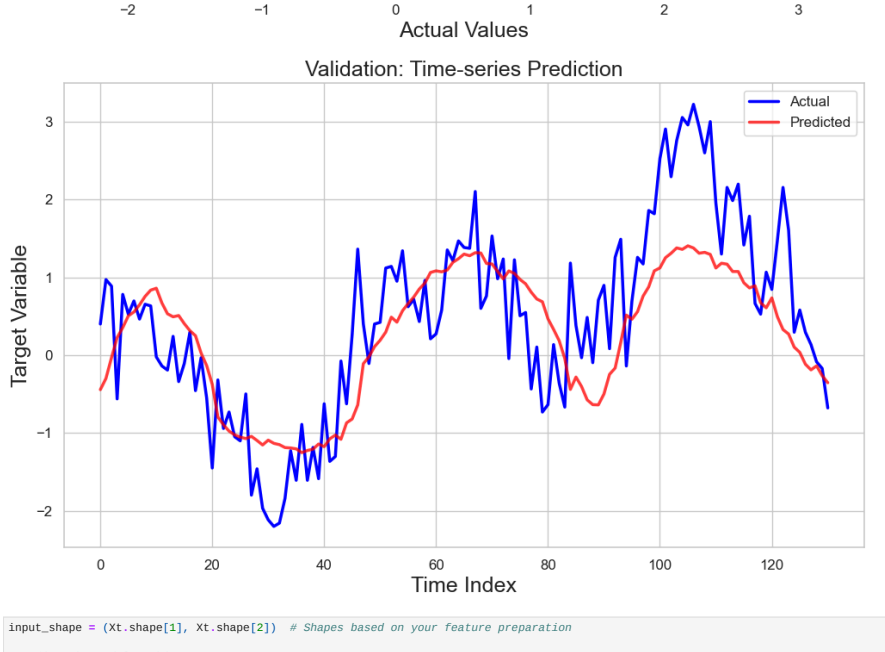
```
19/19 [==============================] - 1s 8ms/step
5/5 [==============================] - 0s 10ms/step
---------- Regression Report ----------
TRAINING:
 MSE: 0.38019180052286855
 MAE: 0.5010044768048829
```


Training Data Parity Plot


Training: Time-series Prediction

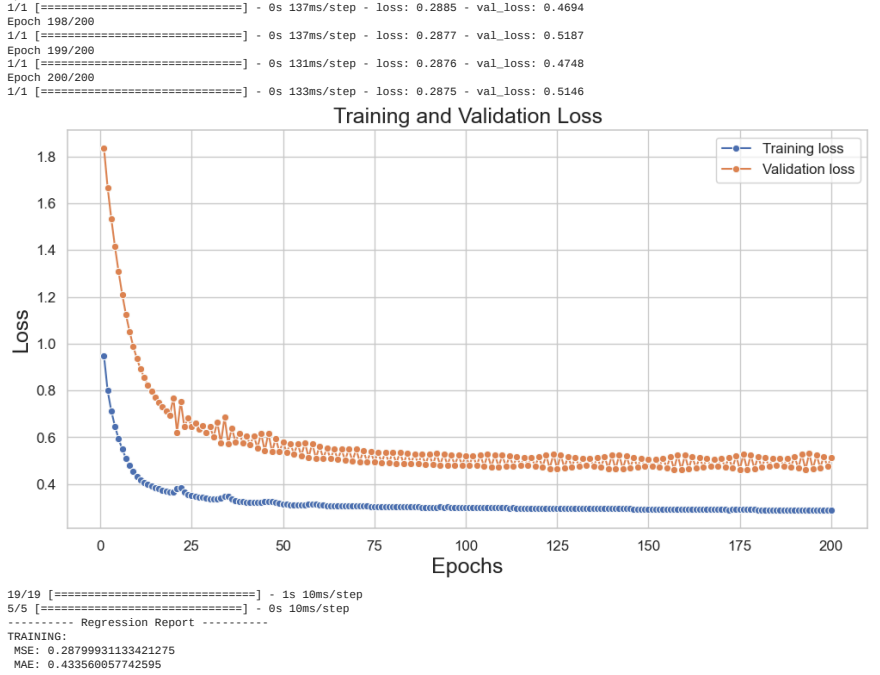
```
VALIDATION:
 MSE: 0.6528944948140699
 MAE: 0.6447353513373126
```


Validation Data Parity Plot

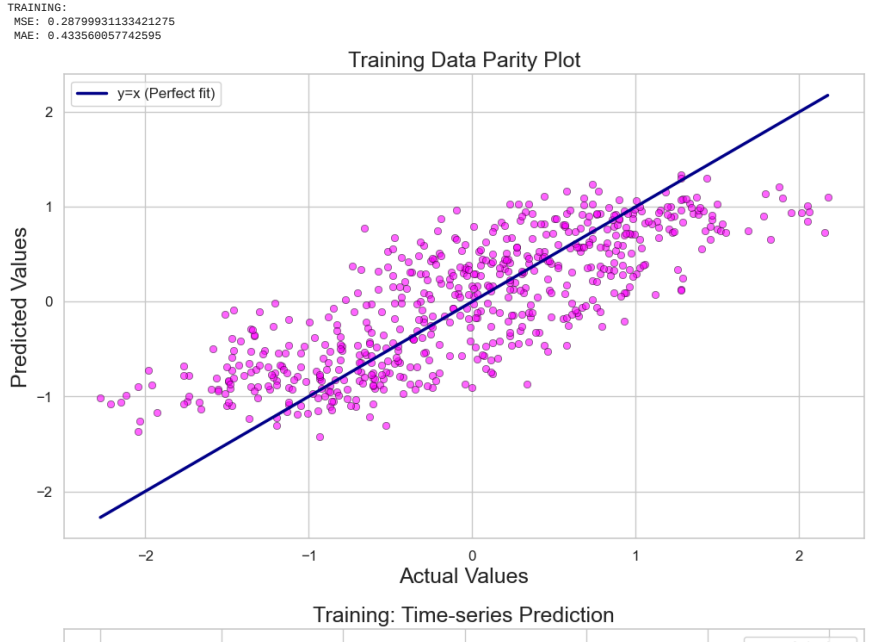

Validation: Time-series Prediction

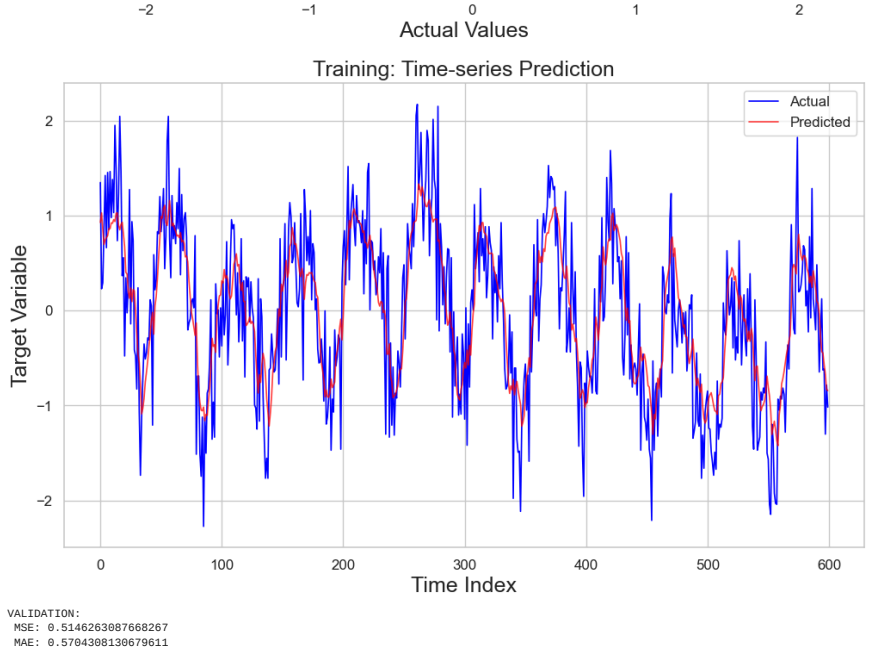
```
In [ ]: input_shape = (Xt.shape[1], Xt.shape[2])  # Shapes based on your feature preparation

        # Define the model architecture
        model_biLSTM = keras.Sequential([
            Bidirectional(LSTM(64, return_sequences=False), input_shape=input_shape),
            Dense(1, activation='linear')
        ])

        # Compile the model
        model_biLSTM.compile(optimizer=keras.optimizers.RMSprop(learning_rate=0.001), loss='mean_squared_error')

        # Display the model summary to verify its structure
        print(model_biLSTM.summary())

        # Train the model
        history_biLSTM = model_biLSTM.fit(Xt, Yt, epochs=200, batch_size=len(Xt), verbose=1, validation_data=(Xv, Yv))


        # Plot training and validation loss
        history_plot(history_biLSTM)

        # Predict and evaluate the model
        Yt_pred_biLSTM = model_biLSTM.predict(Xt)
        Yv_pred_biLSTM = model_biLSTM.predict(Xv)
        regression_report(Yt, Yt_pred_biLSTM, Yv, Yv_pred_biLSTM)
```

```
Model: "sequential_63"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 bidirectional_3 (Bidirecti  (None, 128)               33792
 onal)

 dense_75 (Dense)            (None, 1)                 129

=================================================================
Total params: 33921 (132.50 KB)
Trainable params: 33921 (132.50 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
None
Epoch 1/200
1/1 [==============================] - 13s 13s/step - loss: 0.9480 - val_loss: 1.8351
Epoch 2/200
1/1 [==============================] - 0s 179ms/step - loss: 0.7996 - val_loss: 1.6681
Epoch 3/200
1/1 [==============================] - 0s 143ms/step - loss: 0.7127 - val_loss: 1.5348
Epoch 4/200
1/1 [==============================] - 0s 159ms/step - loss: 0.6473 - val_loss: 1.4153
Epoch 5/200
1/1 [==============================] - 0s 150ms/step - loss: 0.5938 - val_loss: 1.3074
Epoch 6/200
1/1 [==============================] - 0s 153ms/step - loss: 0.5487 - val_loss: 1.2106
Epoch 7/200
1/1 [==============================] - 0s 167ms/step - loss: 0.5105 - val_loss: 1.1255
Epoch 8/200
1/1 [==============================] - 0s 139ms/step - loss: 0.4787 - val_loss: 1.0521
Epoch 9/200
1/1 [==============================] - 0s 133ms/step - loss: 0.4528 - val_loss: 0.9895
Epoch 10/200
1/1 [==============================] - 0s 134ms/step - loss: 0.4324 - val_loss: 0.9365
Epoch 11/200
1/1 [==============================] - 0s 132ms/step - loss: 0.4172 - val_loss: 0.8923
Epoch 12/200
1/1 [==============================] - 0s 154ms/step - loss: 0.4061 - val_loss: 0.8552
Epoch 13/200
1/1 [==============================] - 0s 152ms/step - loss: 0.3978 - val_loss: 0.8237
Epoch 14/200
1/1 [==============================] - 0s 167ms/step - loss: 0.3912 - val_loss: 0.7962
Epoch 15/200
1/1 [==============================] - 0s 157ms/step - loss: 0.3854 - val_loss: 0.7717
Epoch 16/200
1/1 [==============================] - 0s 156ms/step - loss: 0.3799 - val_loss: 0.7495
Epoch 17/200
1/1 [==============================] - 0s 133ms/step - loss: 0.3747 - val_loss: 0.7301
Epoch 18/200
1/1 [==============================] - 0s 133ms/step - loss: 0.3698 - val_loss: 0.7119
Epoch 19/200
1/1 [==============================] - 0s 144ms/step - loss: 0.3652 - val_loss: 0.6948
Epoch 20/200
1/1 [==============================] - 0s 136ms/step - loss: 0.3642 - val_loss: 0.7678
Epoch 21/200
1/1 [==============================] - 0s 248ms/step - loss: 0.3789 - val_loss: 0.6217
Epoch 22/200
1/1 [==============================] - 0s 229ms/step - loss: 0.3855 - val_loss: 0.7544
Epoch 23/200
1/1 [==============================] - 0s 132ms/step - loss: 0.3666 - val_loss: 0.6447
Epoch 24/200
1/1 [==============================] - 0s 135ms/step - loss: 0.3545 - val_loss: 0.6820
Epoch 25/200
1/1 [==============================] - 0s 134ms/step - loss: 0.3500 - val_loss: 0.6463
Epoch 26/200
1/1 [==============================] - 0s 144ms/step - loss: 0.3471 - val_loss: 0.6599
Epoch 27/200
1/1 [==============================] - 0s 146ms/step - loss: 0.3445 - val_loss: 0.6359
Epoch 28/200
1/1 [==============================] - 0s 190ms/step - loss: 0.3421 - val_loss: 0.6482
Epoch 29/200
1/1 [==============================] - 0s 187ms/step - loss: 0.3398 - val_loss: 0.6218
Epoch 30/200
1/1 [==============================] - 0s 200ms/step - loss: 0.3377 - val_loss: 0.6453
Epoch 31/200
1/1 [==============================] - 0s 197ms/step - loss: 0.3361 - val_loss: 0.6009
Epoch 32/200
1/1 [==============================] - 0s 159ms/step - loss: 0.3357 - val_loss: 0.6658
Epoch 33/200
1/1 [==============================] - 0s 147ms/step - loss: 0.3389 - val_loss: 0.5755
Epoch 34/200
1/1 [==============================] - 0s 132ms/step - loss: 0.3458 - val_loss: 0.6858
Epoch 35/200
1/1 [==============================] - 0s 129ms/step - loss: 0.3475 - val_loss: 0.5733
Epoch 36/200
1/1 [==============================] - 0s 135ms/step - loss: 0.3368 - val_loss: 0.6395
Epoch 37/200
1/1 [==============================] - 0s 141ms/step - loss: 0.3306 - val_loss: 0.5800
Epoch 38/200
1/1 [==============================] - 0s 150ms/step - loss: 0.3264 - val_loss: 0.6164
Epoch 39/200
1/1 [==============================] - 0s 168ms/step - loss: 0.3241 - val_loss: 0.5772
Epoch 40/200
1/1 [==============================] - 0s 130ms/step - loss: 0.3223 - val_loss: 0.6070
Epoch 41/200
1/1 [==============================] - 0s 133ms/step - loss: 0.3211 - val_loss: 0.5683
Epoch 42/200
1/1 [==============================] - 0s 137ms/step - loss: 0.3202 - val_loss: 0.6067
Epoch 43/200
1/1 [==============================] - 0s 134ms/step - loss: 0.3202 - val_loss: 0.5549
Epoch 44/200
1/1 [==============================] - 0s 132ms/step - loss: 0.3211 - val_loss: 0.6172
Epoch 45/200
1/1 [==============================] - 0s 139ms/step - loss: 0.3240 - val_loss: 0.5420
Epoch 46/200
1/1 [==============================] - 0s 135ms/step - loss: 0.3252 - val_loss: 0.6180
Epoch 47/200
1/1 [==============================] - 0s 174ms/step - loss: 0.3252 - val_loss: 0.5373
Epoch 48/200
1/1 [==============================] - 0s 151ms/step - loss: 0.3202 - val_loss: 0.5959
Epoch 49/200
1/1 [==============================] - 0s 150ms/step - loss: 0.3172 - val_loss: 0.5376
Epoch 50/200
1/1 [==============================] - 0s 162ms/step - loss: 0.3142 - val_loss: 0.5802
Epoch 51/200
1/1 [==============================] - 0s 131ms/step - loss: 0.3127 - val_loss: 0.5349
Epoch 52/200
1/1 [==============================] - 0s 146ms/step - loss: 0.3115 - val_loss: 0.5732
Epoch 53/200
1/1 [==============================] - 0s 131ms/step - loss: 0.3110 - val_loss: 0.5287
Epoch 54/200
1/1 [==============================] - 0s 264ms/step - loss: 0.3107 - val_loss: 0.5729
Epoch 55/200
1/1 [==============================] - 0s 271ms/step - loss: 0.3114 - val_loss: 0.5298
Epoch 56/200
1/1 [==============================] - 0s 145ms/step - loss: 0.3121 - val_loss: 0.5766
Epoch 57/200
1/1 [==============================] - 0s 168ms/step - loss: 0.3138 - val_loss: 0.5140
Epoch 58/200
1/1 [==============================] - 0s 154ms/step - loss: 0.3135 - val_loss: 0.5737
Epoch 59/200
1/1 [==============================] - 0s 134ms/step - loss: 0.3136 - val_loss: 0.5107
Epoch 60/200
1/1 [==============================] - 0s 128ms/step - loss: 0.3110 - val_loss: 0.5624
Epoch 61/200
1/1 [==============================] - 0s 135ms/step - loss: 0.3097 - val_loss: 0.5097
Epoch 62/200
1/1 [==============================] - 0s 135ms/step - loss: 0.3077 - val_loss: 0.5533
Epoch 63/200
1/1 [==============================] - 0s 144ms/step - loss: 0.3070 - val_loss: 0.5080
Epoch 64/200
1/1 [==============================] - 0s 153ms/step - loss: 0.3061 - val_loss: 0.5492
Epoch 65/200
1/1 [==============================] - 0s 138ms/step - loss: 0.3061 - val_loss: 0.5047
Epoch 66/200
1/1 [==============================] - 0s 162ms/step - loss: 0.3059 - val_loss: 0.5492
Epoch 67/200
1/1 [==============================] - 0s 143ms/step - loss: 0.3066 - val_loss: 0.5007
Epoch 68/200
1/1 [==============================] - 0s 126ms/step - loss: 0.3068 - val_loss: 0.5507
Epoch 69/200
1/1 [==============================] - 0s 131ms/step - loss: 0.3079 - val_loss: 0.4973
Epoch 70/200
1/1 [==============================] - 0s 136ms/step - loss: 0.3074 - val_loss: 0.5488
Epoch 71/200
1/1 [==============================] - 0s 132ms/step - loss: 0.3077 - val_loss: 0.4954
Epoch 72/200
1/1 [==============================] - 0s 157ms/step - loss: 0.3061 - val_loss: 0.5433
Epoch 73/200
1/1 [==============================] - 0s 129ms/step - loss: 0.3057 - val_loss: 0.4945
Epoch 74/200
1/1 [==============================] - 0s 322ms/step - loss: 0.3043 - val_loss: 0.5384
Epoch 75/200
1/1 [==============================] - 0s 209ms/step - loss: 0.3040 - val_loss: 0.4935
Epoch 76/200
1/1 [==============================] - 0s 121ms/step - loss: 0.3032 - val_loss: 0.5362
Epoch 77/200
1/1 [==============================] - 0s 127ms/step - loss: 0.3033 - val_loss: 0.4917
Epoch 78/200
1/1 [==============================] - 0s 162ms/step - loss: 0.3030 - val_loss: 0.5363
Epoch 79/200
1/1 [==============================] - 0s 140ms/step - loss: 0.3036 - val_loss: 0.4897
Epoch 80/200
1/1 [==============================] - 0s 175ms/step - loss: 0.3034 - val_loss: 0.5369
Epoch 81/200
1/1 [==============================] - 0s 146ms/step - loss: 0.3041 - val_loss: 0.4881
Epoch 82/200
1/1 [==============================] - 0s 232ms/step - loss: 0.3037 - val_loss: 0.5355
Epoch 83/200
1/1 [==============================] - 0s 163ms/step - loss: 0.3040 - val_loss: 0.4873
Epoch 84/200
1/1 [==============================] - 0s 172ms/step - loss: 0.3030 - val_loss: 0.5324
Epoch 85/200
1/1 [==============================] - 0s 148ms/step - loss: 0.3029 - val_loss: 0.4866
Epoch 86/200
1/1 [==============================] - 0s 124ms/step - loss: 0.3018 - val_loss: 0.5297
Epoch 87/200
1/1 [==============================] - 0s 128ms/step - loss: 0.3017 - val_loss: 0.4856
Epoch 88/200
1/1 [==============================] - 0s 129ms/step - loss: 0.3010 - val_loss: 0.5288
Epoch 89/200
1/1 [==============================] - 0s 133ms/step - loss: 0.3012 - val_loss: 0.4840
Epoch 90/200
1/1 [==============================] - 0s 128ms/step - loss: 0.3008 - val_loss: 0.5294
Epoch 91/200
1/1 [==============================] - 0s 131ms/step - loss: 0.3012 - val_loss: 0.4823
Epoch 92/200
1/1 [==============================] - 0s 131ms/step - loss: 0.3009 - val_loss: 0.5299
Epoch 93/200
1/1 [==============================] - 0s 129ms/step - loss: 0.3014 - val_loss: 0.4813
Epoch 94/200
1/1 [==============================] - 0s 133ms/step - loss: 0.3010 - val_loss: 0.5287
Epoch 95/200
1/1 [==============================] - 0s 134ms/step - loss: 0.3013 - val_loss: 0.4813
Epoch 96/200
1/1 [==============================] - 0s 138ms/step - loss: 0.3006 - val_loss: 0.5258
Epoch 97/200
1/1 [==============================] - 0s 138ms/step - loss: 0.3007 - val_loss: 0.4818
Epoch 98/200
1/1 [==============================] - 0s 143ms/step - loss: 0.2998 - val_loss: 0.5230
Epoch 99/200
1/1 [==============================] - 0s 175ms/step - loss: 0.2998 - val_loss: 0.4817
Epoch 100/200
1/1 [==============================] - 0s 166ms/step - loss: 0.2992 - val_loss: 0.5217
Epoch 101/200
1/1 [==============================] - 0s 147ms/step - loss: 0.2993 - val_loss: 0.4807
Epoch 102/200
1/1 [==============================] - 0s 144ms/step - loss: 0.2988 - val_loss: 0.5223
Epoch 103/200
1/1 [==============================] - 0s 125ms/step - loss: 0.2991 - val_loss: 0.4785
Epoch 104/200
1/1 [==============================] - 0s 131ms/step - loss: 0.2988 - val_loss: 0.5244
Epoch 105/200
1/1 [==============================] - 0s 132ms/step - loss: 0.2992 - val_loss: 0.4759
Epoch 106/200
1/1 [==============================] - 0s 148ms/step - loss: 0.2988 - val_loss: 0.5261
Epoch 107/200
1/1 [==============================] - 0s 134ms/step - loss: 0.2992 - val_loss: 0.4740
Epoch 108/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2986 - val_loss: 0.5258
Epoch 109/200
1/1 [==============================] - 0s 139ms/step - loss: 0.2988 - val_loss: 0.4738
Epoch 110/200
1/1 [==============================] - 0s 188ms/step - loss: 0.2981 - val_loss: 0.5230
Epoch 111/200
1/1 [==============================] - 0s 146ms/step - loss: 0.2982 - val_loss: 0.4752
Epoch 112/200
1/1 [==============================] - 0s 164ms/step - loss: 0.2976 - val_loss: 0.5192
Epoch 113/200
1/1 [==============================] - 0s 150ms/step - loss: 0.2977 - val_loss: 0.4777
Epoch 114/200
1/1 [==============================] - 0s 125ms/step - loss: 0.2973 - val_loss: 0.5153
Epoch 115/200
1/1 [==============================] - 0s 129ms/step - loss: 0.2976 - val_loss: 0.4803
Epoch 116/200
1/1 [==============================] - 0s 141ms/step - loss: 0.2972 - val_loss: 0.5128
Epoch 117/200
1/1 [==============================] - 0s 159ms/step - loss: 0.2974 - val_loss: 0.4805
Epoch 118/200
1/1 [==============================] - 0s 142ms/step - loss: 0.2969 - val_loss: 0.5131
Epoch 119/200
1/1 [==============================] - 0s 171ms/step - loss: 0.2968 - val_loss: 0.4774
Epoch 120/200
1/1 [==============================] - 0s 146ms/step - loss: 0.2963 - val_loss: 0.5167
Epoch 121/200
1/1 [==============================] - 0s 126ms/step - loss: 0.2964 - val_loss: 0.4721
Epoch 122/200
1/1 [==============================] - 0s 126ms/step - loss: 0.2962 - val_loss: 0.5232
Epoch 123/200
1/1 [==============================] - 0s 130ms/step - loss: 0.2966 - val_loss: 0.4670
Epoch 124/200
1/1 [==============================] - 0s 128ms/step - loss: 0.2965 - val_loss: 0.5274
Epoch 125/200
1/1 [==============================] - 0s 132ms/step - loss: 0.2969 - val_loss: 0.4663
Epoch 126/200
1/1 [==============================] - 0s 141ms/step - loss: 0.2963 - val_loss: 0.5242
Epoch 127/200
1/1 [==============================] - 0s 132ms/step - loss: 0.2963 - val_loss: 0.4695
Epoch 128/200
1/1 [==============================] - 0s 136ms/step - loss: 0.2955 - val_loss: 0.5181
Epoch 129/200
1/1 [==============================] - 0s 131ms/step - loss: 0.2956 - val_loss: 0.4735
Epoch 130/200
1/1 [==============================] - 0s 138ms/step - loss: 0.2952 - val_loss: 0.5129
Epoch 131/200
1/1 [==============================] - 0s 128ms/step - loss: 0.2956 - val_loss: 0.4773
Epoch 132/200
1/1 [==============================] - 0s 132ms/step - loss: 0.2954 - val_loss: 0.5097
Epoch 133/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2956 - val_loss: 0.4784
Epoch 134/200
1/1 [==============================] - 0s 171ms/step - loss: 0.2951 - val_loss: 0.5094
Epoch 135/200
1/1 [==============================] - 0s 180ms/step - loss: 0.2950 - val_loss: 0.4760
Epoch 136/200
1/1 [==============================] - 0s 160ms/step - loss: 0.2944 - val_loss: 0.5118
Epoch 137/200
1/1 [==============================] - 0s 187ms/step - loss: 0.2943 - val_loss: 0.4719
Epoch 138/200
1/1 [==============================] - 0s 162ms/step - loss: 0.2940 - val_loss: 0.5167
Epoch 139/200
1/1 [==============================] - 0s 149ms/step - loss: 0.2943 - val_loss: 0.4671
Epoch 140/200
1/1 [==============================] - 0s 130ms/step - loss: 0.2942 - val_loss: 0.5225
Epoch 141/200
1/1 [==============================] - 0s 141ms/step - loss: 0.2947 - val_loss: 0.4642
Epoch 142/200
1/1 [==============================] - 0s 135ms/step - loss: 0.2945 - val_loss: 0.5236
Epoch 143/200
1/1 [==============================] - 0s 127ms/step - loss: 0.2947 - val_loss: 0.4653
Epoch 144/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2940 - val_loss: 0.5193
Epoch 145/200
1/1 [==============================] - 0s 135ms/step - loss: 0.2940 - val_loss: 0.4688
Epoch 146/200
1/1 [==============================] - 0s 136ms/step - loss: 0.2934 - val_loss: 0.5138
Epoch 147/200
1/1 [==============================] - 0s 136ms/step - loss: 0.2936 - val_loss: 0.4728
Epoch 148/200
1/1 [==============================] - 0s 136ms/step - loss: 0.2933 - val_loss: 0.5093
Epoch 149/200
1/1 [==============================] - 0s 141ms/step - loss: 0.2936 - val_loss: 0.4763
Epoch 150/200
1/1 [==============================] - 0s 137ms/step - loss: 0.2933 - val_loss: 0.5069
Epoch 151/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2934 - val_loss: 0.4767
Epoch 152/200
1/1 [==============================] - 0s 142ms/step - loss: 0.2929 - val_loss: 0.5074
Epoch 153/200
1/1 [==============================] - 0s 143ms/step - loss: 0.2927 - val_loss: 0.4739
Epoch 154/200
1/1 [==============================] - 0s 228ms/step - loss: 0.2922 - val_loss: 0.5110
Epoch 155/200
1/1 [==============================] - 0s 199ms/step - loss: 0.2923 - val_loss: 0.4690
Epoch 156/200
1/1 [==============================] - 0s 163ms/step - loss: 0.2921 - val_loss: 0.5181
Epoch 157/200
1/1 [==============================] - 0s 159ms/step - loss: 0.2926 - val_loss: 0.4634
Epoch 158/200
1/1 [==============================] - 0s 147ms/step - loss: 0.2927 - val_loss: 0.5247
Epoch 159/200
1/1 [==============================] - 0s 122ms/step - loss: 0.2932 - val_loss: 0.4616
Epoch 160/200
1/1 [==============================] - 0s 131ms/step - loss: 0.2927 - val_loss: 0.5232
Epoch 161/200
1/1 [==============================] - 0s 130ms/step - loss: 0.2926 - val_loss: 0.4647
Epoch 162/200
1/1 [==============================] - 0s 134ms/step - loss: 0.2918 - val_loss: 0.5168
Epoch 163/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2917 - val_loss: 0.4692
Epoch 164/200
1/1 [==============================] - 0s 129ms/step - loss: 0.2913 - val_loss: 0.5114
Epoch 165/200
1/1 [==============================] - 0s 141ms/step - loss: 0.2915 - val_loss: 0.4738
Epoch 166/200
1/1 [==============================] - 0s 141ms/step - loss: 0.2915 - val_loss: 0.5078
Epoch 167/200
1/1 [==============================] - 0s 137ms/step - loss: 0.2919 - val_loss: 0.4771
Epoch 168/200
1/1 [==============================] - 0s 146ms/step - loss: 0.2917 - val_loss: 0.5065
Epoch 169/200
1/1 [==============================] - 0s 148ms/step - loss: 0.2916 - val_loss: 0.4765
Epoch 170/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2910 - val_loss: 0.5078
Epoch 171/200
1/1 [==============================] - 0s 132ms/step - loss: 0.2907 - val_loss: 0.4730
Epoch 172/200
1/1 [==============================] - 0s 136ms/step - loss: 0.2902 - val_loss: 0.5125
Epoch 173/200
1/1 [==============================] - 0s 180ms/step - loss: 0.2903 - val_loss: 0.4675
Epoch 174/200
1/1 [==============================] - 0s 170ms/step - loss: 0.2903 - val_loss: 0.5211
Epoch 175/200
1/1 [==============================] - 0s 147ms/step - loss: 0.2909 - val_loss: 0.4621
Epoch 176/200
1/1 [==============================] - 0s 143ms/step - loss: 0.2910 - val_loss: 0.5271
Epoch 177/200
1/1 [==============================] - 0s 143ms/step - loss: 0.2914 - val_loss: 0.4619
Epoch 178/200
1/1 [==============================] - 0s 144ms/step - loss: 0.2907 - val_loss: 0.5234
Epoch 179/200
1/1 [==============================] - 0s 125ms/step - loss: 0.2906 - val_loss: 0.4662
Epoch 180/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2898 - val_loss: 0.5166
Epoch 181/200
1/1 [==============================] - 0s 175ms/step - loss: 0.2897 - val_loss: 0.4711
Epoch 182/200
1/1 [==============================] - 0s 140ms/step - loss: 0.2894 - val_loss: 0.5118
Epoch 183/200
1/1 [==============================] - 0s 148ms/step - loss: 0.2898 - val_loss: 0.4760
Epoch 184/200
1/1 [==============================] - 0s 142ms/step - loss: 0.2898 - val_loss: 0.5088
Epoch 185/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2902 - val_loss: 0.4790
Epoch 186/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2899 - val_loss: 0.5080
Epoch 187/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2896 - val_loss: 0.4779
Epoch 188/200
1/1 [==============================] - 0s 129ms/step - loss: 0.2890 - val_loss: 0.5100
Epoch 189/200
1/1 [==============================] - 0s 138ms/step - loss: 0.2886 - val_loss: 0.4740
Epoch 190/200
1/1 [==============================] - 0s 141ms/step - loss: 0.2883 - val_loss: 0.5164
Epoch 191/200
1/1 [==============================] - 0s 153ms/step - loss: 0.2885 - val_loss: 0.4678
Epoch 192/200
1/1 [==============================] - 0s 163ms/step - loss: 0.2886 - val_loss: 0.5269
Epoch 193/200
1/1 [==============================] - 0s 160ms/step - loss: 0.2894 - val_loss: 0.4626
Epoch 194/200
1/1 [==============================] - 0s 145ms/step - loss: 0.2894 - val_loss: 0.5319
Epoch 195/200
1/1 [==============================] - 0s 146ms/step - loss: 0.2897 - val_loss: 0.4641
Epoch 196/200
1/1 [==============================] - 0s 126ms/step - loss: 0.2888 - val_loss: 0.5257
Epoch 197/200
1/1 [==============================] - 0s 137ms/step - loss: 0.2885 - val_loss: 0.4694
Epoch 198/200
1/1 [==============================] - 0s 137ms/step - loss: 0.2877 - val_loss: 0.5187
Epoch 199/200
1/1 [==============================] - 0s 131ms/step - loss: 0.2876 - val_loss: 0.4748
Epoch 200/200
1/1 [==============================] - 0s 133ms/step - loss: 0.2875 - val_loss: 0.5146
```

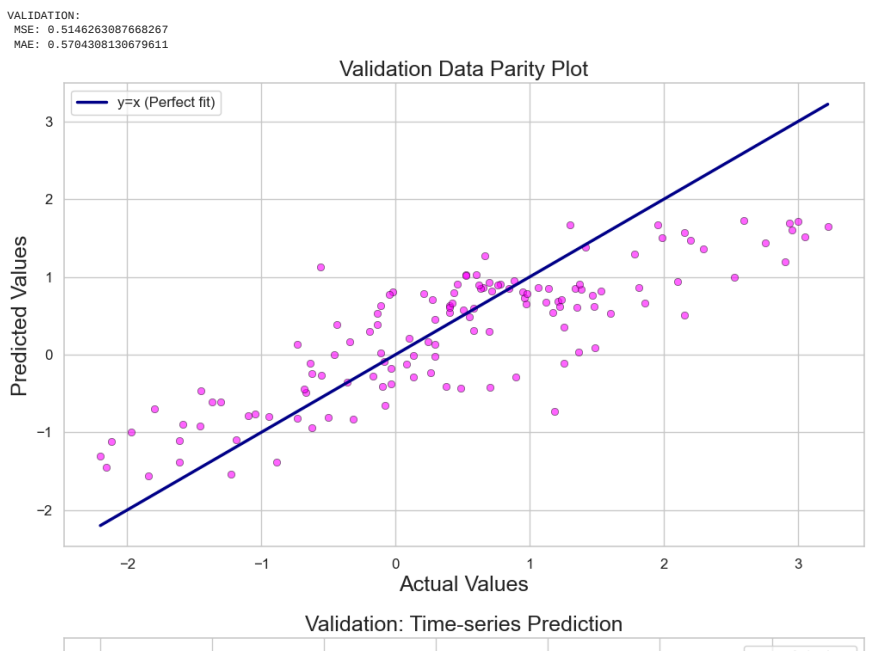

Training and Validation Loss

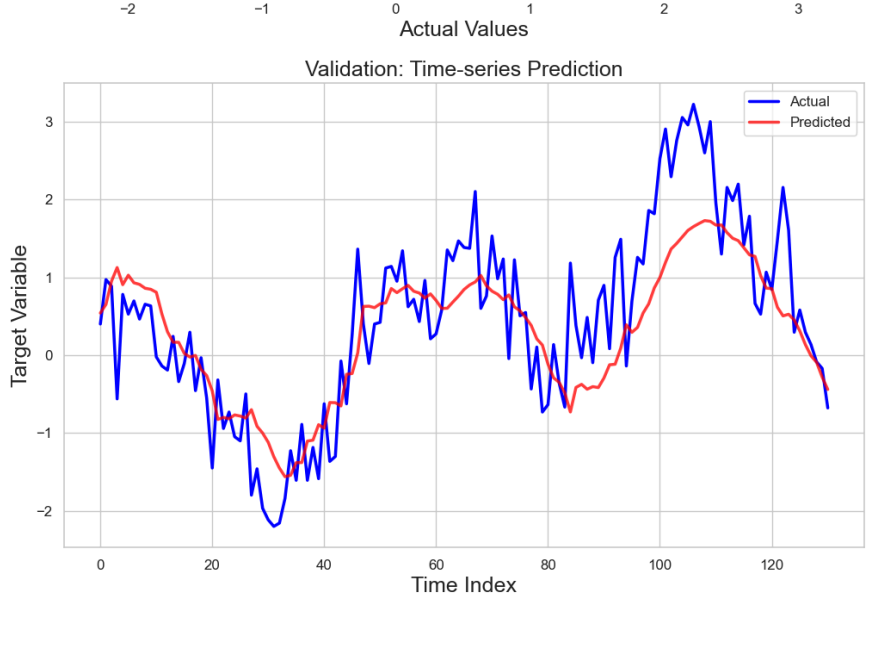
```
19/19 [==============================] - 1s 10ms/step
5/5 [==============================] - 0s 10ms/step
---------- Regression Report ----------
TRAINING:
 MSE: 0.28799931133421275
 MAE: 0.433560057742595
```


Training Data Parity Plot


Training: Time-series Prediction

```
VALIDATION:
 MSE: 0.5146263087668267
 MAE: 0.5704308130679611
```


Validation Data Parity Plot


Validation: Time-series Prediction